

PROGRAMMING FUNDAMENTALS

The Building Blocks Every Coder Must Know

GCD · LCM · Prime Check · Sieve · Reverse · Sum of Digits
Armstrong · Palindrome · Perfect · Strong · Automorphic

C++ · Java · Python



Why Fundamentals Matter

GCD / LCM

Simplifying fractions, scheduling, range-based array problems

Reverse / Palindrome

Number manipulation, string problems, two-pointer patterns

Number Properties

Perfect, Strong, Automorphic — loop logic and pattern building

Prime Numbers

Sieve for range queries, factorization, cryptography basics

Digit Operations

Sum of digits, digit count — used in hashing, magic numbers

Interview Reality

Asked in pre-screens, campus drives, and warm-up coding rounds

01 · GCD — Greatest Common Divisor

EXAMPLE

```
gcd(48, 18)
step1: a=48, b=18 → a=18, b=48%18=12
step2: a=18, b=12 → a=12, b=18%12=6
step3: a=12, b=6 → a=6, b=12%6=0
b == 0 → return a = 6 ✓
```

💡 KEY INSIGHT

Euclidean: $\text{gcd}(a,b) = \text{gcd}(b, a\%b)$
Base case: when $b = 0$, return a
Each step shrinks the problem
Use iterative — no recursion needed

📦 Time: $O(\log \min(a,b))$ 📦 Space: $O(1)$

PSEUDOCODE

```
function gcd(a, b):
    while b != 0:
        temp = b
        b = a % b
        a = temp
    return a
```

```
// Trace: gcd(48, 18)
// a=48, b=18 → b=48%18=12, a=18
// a=18, b=12 → b=18%12=6, a=12
// a=12, b=6 → b=12%6=0, a=6
// b=0 → return 6 ✓
```

```
// GCD of array:
```

```
function gcdArray(arr):
    result = arr[0]
    for i = 1 to len-1:
        result = gcd(result, arr[i])
    return result
```

GCD — Greatest Common Divisor — C++ · Java · Python

C++

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// GCD of array
int gcdArray(vector<int>& arr) {
    int result = arr[0];
    for (int i = 1; i < arr.size();
i++)
        result = gcd(result,
arr[i]);
    return result;
}

// C++17 built-in:
// #include <numeric>
// __gcd(a, b)
```

Java

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// GCD of array
int gcdArray(int[] arr) {
    int result = arr[0];
    for (int i = 1; i < arr.length;
i++)
        result = gcd(result,
arr[i]);
    return result;
}

// Usage:
// int g = gcd(48, 18); // 6
```

Python

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# GCD of array
def gcd_array(arr):
    result = arr[0]
    for i in range(1, len(arr)):
        result = gcd(result,
arr[i])
    return result

# Built-in:
import math
math.gcd(48, 18)    # 6
```

02 · LCM — Least Common Multiple

EXAMPLE

```
lcm(12, 18)
gcd(12, 18) = 6
lcm = (12 / 6) * 18
      = 2 * 18
      = 36 ✓
```

💡 KEY INSIGHT

Formula: $\text{lcm}(a,b) = (a / \text{gcd}(a,b)) * b$
Divide FIRST then multiply — avoids overflow
Never write $a*b/\text{gcd}$ — $a*b$ can overflow int
LCM of array: apply lcm pair by pair

🕒 Time: $O(\log \min(a,b))$ 📦 Space: $O(1)$

PSEUDOCODE

```
function lcm(a, b):
    g = gcd(a, b)
    return (a / g) * b

// ⚠️ Do NOT write a*b/gcd(a,b)
//    a*b can overflow int!
//    Divide first, then multiply.

// Trace: lcm(12, 18)
// gcd(12,18) = 6
// (12 / 6) * 18 = 2 * 18 = 36 ✓

// LCM of array:
function lcmArray(arr):
    result = arr[0]
    for i = 1 to len-1:
        result = lcm(result, arr[i])
    return result
```

LCM — Least Common Multiple — C++ · Java · Python

C++

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Divide first to avoid overflow
long long lcm(long long a, long long b) {
    return (a / gcd(a, b)) * b;
}

// LCM of array
long long lcmArray(vector<int>& arr) {
    long long result = arr[0];
    for (int i = 1; i < arr.size(); i++)
        result = lcm(result, arr[i]);
    return result;
}
```

Java

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Divide first to avoid overflow
long lcm(long a, long b) {
    return (a / gcd((int)a, (int)b)) * b;
}

// LCM of array
long lcmArray(int[] arr) {
    long result = arr[0];
    for (int i = 1; i < arr.length; i++)
        result = lcm(result, arr[i]);
    return result;
}
```

Python

```
import math

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# Divide first to avoid overflow
def lcm(a, b):
    return (a // gcd(a, b)) * b

# LCM of array
def lcm_array(arr):
    result = arr[0]
    for i in range(1, len(arr)):
        result = lcm(result, arr[i])
    return result

# Python 3.9+: math.lcm(a, b)
```

03 · Prime Number Check

EXAMPLE

```
isPrime(29)?  
Check i = 2 to  $\sqrt{29} \approx 5$   
i=2:  $29\%2 \neq 0$   
i=3:  $29\%3 \neq 0$   
i=4:  $29\%4 \neq 0$   
i=5:  $29\%5 \neq 0 \rightarrow \text{PRIME} \checkmark$ 
```

💡 KEY INSIGHT

A prime has exactly 2 factors: 1 and itself
Brute force: check $2..n-1 \rightarrow O(n)$ ✗
Optimized: check only $2..\sqrt{n} \rightarrow O(\sqrt{n})$ ✓
Why $\sqrt{n}$? Factors come in pairs: if $a*b=n$
and $a > \sqrt{n}$, then $b < \sqrt{n}$ — already checked

🕒 Time: $O(\sqrt{n})$ 📦 Space: $O(1)$

PSEUDOCODE

```
function isPrime(n):  
    if n < 2: return False  
    if n == 2: return True  
  
    // Check from 2 up to  $\sqrt{n}$   
    i = 2  
    while i * i <= n:  
        if n % i == 0:  
            return False ← not prime  
        i = i + 1  
  
    return True ← prime!  
  
// Why  $i*i \leq n$  not  $i \leq \text{sqrt}(n)$ ?  
// Avoids float sqrt imprecision  
//  $i*i \leq n$  is exact integer check  
  
// isPrime(1) → False  
// isPrime(2) → True  
// isPrime(29) → True
```

Prime Number Check — C++ · Java · Python

C++

```
bool isPrime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;

    for (int i = 2; (long long)i *
i <= n; i++) {
        if (n % i == 0)
            return false;
    }
    return true;
}

// Usage:
// isPrime(1)  → false
// isPrime(2)  → true
// isPrime(29) → true
// isPrime(25) → false
```

Java

```
boolean isPrime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;

    for (int i = 2; (long) i * i <=
n; i++) {
        if (n % i == 0)
            return false;
    }
    return true;
}

// Usage:
// isPrime(1)  → false
// isPrime(2)  → true
// isPrime(29) → true
// isPrime(25) → false
```

Python

```
def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True

    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True

# is_prime(1)  → False
# is_prime(2)  → True
# is_prime(29) → True
# is_prime(25) → False
```


04 · Sieve of Eratosthenes

EXAMPLE

Find all primes up to 10:
Mark: [T,T,T,T,T,T,T,T,T,T]
p=2: mark 4,6,8,10 ✗
p=3: mark 9 ✗
p=4: already marked, skip
Primes = {2, 3, 5, 7} ✓

💡 KEY INSIGHT

Best algorithm to find ALL primes up to N
Mark all multiples of each prime as not prime
Inner loop starts at $p \cdot p$ — smaller multiples
already marked by earlier primes
Time: $O(N \log \log N)$ — very fast in practice

📦 Time: $O(N \log \log N)$ 📦 Space: $O(N)$

PSEUDOCODE

```
function sieve(n):  
    is_prime = [True] * (n + 1)  
    is_prime[0] = False  
    is_prime[1] = False  
  
    p = 2  
    while p * p <= n:  
        if is_prime[p]:  
            // Mark multiples from p*p  
            j = p * p  
            while j <= n:  
                is_prime[j] = False  
                j = j + p  
            p = p + 1  
  
    return is_prime[]  
  
// Why start at p*p?  
// 2p, 3p ... already marked by smaller primes
```

Sieve of Eratosthenes — C++ · Java · Python

C++

```
vector<bool> sieve(int n) {
    vector<bool> isPrime(n + 1,
true);
    isPrime[0] = isPrime[1] =
false;

    for (int p = 2; (long long)p *
p <= n; p++) {
        if (isPrime[p]) {
            for (int j = p * p; j
<= n; j += p)
                isPrime[j] = false;
        }
    }
    return isPrime;
}

// Print all primes:
// for (int i = 2; i <= n; i++)
//     if (isPrime[i])
//         cout << i << " ";
```

Java

```
boolean[] sieve(int n) {
    boolean[] isPrime = new
boolean[n + 1];
    Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] =
false;

    for (int p = 2; (long) p * p <=
n; p++) {
        if (isPrime[p]) {
            for (int j = p * p; j
<= n; j += p)
                isPrime[j] = false;
        }
    }
    return isPrime;
}

// Print all primes:
// for (int i = 2; i <= n; i++)
//     if (isPrime[i])
//         System.out.print(i + "
");
```

Python

```
def sieve(n):
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] =
False

    p = 2
    while p * p <= n:
        if is_prime[p]:
            j = p * p
            while j <= n:
                is_prime[j] = False
                j += p
        p += 1

    return is_prime

# Get all primes as list:
# [i for i in range(2, n+1) if
is_prime[i]]
```

05 · Reverse a Number

EXAMPLE

```
reverse(1234)
```

Step 1: digit=4, rev=4, n=123

Step 2: digit=3, rev=43, n=12

Step 3: digit=2, rev=432, n=1

Step 4: digit=1, rev=4321, n=0

Result = 4321 ✓

💡 KEY INSIGHT

Extract last digit: $\text{digit} = n \% 10$

Append to reversed: $\text{rev} = \text{rev} * 10 + \text{digit}$

Remove last digit: $n = n / 10$

Repeat until n becomes 0

Handle negatives: store sign, work on $\text{abs}(n)$

🕒 Time: $O(\log n)$ 📦 Space: $O(1)$

PSEUDOCODE

```
function reverse(n):
```

```
    rev = 0
```

```
    sign = 1
```

```
    if n < 0:
```

```
        sign = -1
```

```
        n = -n
```

```
    while n != 0:
```

```
        digit = n % 10
```

```
        rev = rev * 10 + digit
```

```
        n = n / 10
```

```
    return sign * rev
```

```
// Trace: n = 1234
```

```
// d=4 → rev=4, n=123
```

```
// d=3 → rev=43, n=12
```

```
// d=2 → rev=432, n=1
```

```
// d=1 → rev=4321, n=0 done ✓
```

Reverse a Number — C++ · Java · Python

C++

```
long reverse(long n) {
    long rev = 0;
    int sign = (n < 0) ? -1 : 1;
    n = Math.abs(n); // use abs(n)
    for C++

    while (n != 0) {
        long digit = n % 10;
        rev = rev * 10 + digit;
        n = n / 10;
    }
    return sign * rev;
}

// n = 1234 → 4321
// n = -120 → -21
```

Java

```
long reverse(long n) {
    long rev = 0;
    int sign = (n < 0) ? -1 : 1;
    n = Math.abs(n);

    while (n != 0) {
        long digit = n % 10;
        rev = rev * 10 + digit;
        n = n / 10;
    }
    return sign * rev;
}

// n = 1234 → 4321
// n = -120 → -21
```

Python

```
def reverse_num(n):
    rev = 0
    sign = -1 if n < 0 else 1
    n = abs(n)

    while n != 0:
        digit = n % 10
        rev = rev * 10 + digit
        n = n // 10

    return sign * rev

# reverse_num(1234) → 4321
# reverse_num(-120) → -21
```

06 · Sum of Digits & Count Digits

EXAMPLE

```
sumDigits(1234)
4 + 3 + 2 + 1 = 10 ✓
```

```
countDigits(1234)
1234 → 123 → 12 → 1 → 0
count = 4 ✓
```

💡 KEY INSIGHT

Sum: extract digit with % 10, add to total
Count: just increment count each iteration
Both use same while-loop skeleton
Remove last digit with integer division / 10

🕒 Time: $O(\log n)$ 📦 Space: $O(1)$

PSEUDOCODE

```
// SUM OF DIGITS
```

```
function sumDigits(n):
```

```
    n      = abs(n)
```

```
    total = 0
```

```
    while n != 0:
```

```
        total = total + (n % 10)
```

```
        n      = n / 10
```

```
    return total
```

```
// COUNT DIGITS
```

```
function countDigits(n):
```

```
    if n == 0: return 1
```

```
    n      = abs(n)
```

```
    count = 0
```

```
    while n != 0:
```

```
        count = count + 1
```

Sum of Digits & Count Digits — C++ · Java · Python

C++

```
// Sum of digits
int sumDigits(int n) {
    n = abs(n);
    int total = 0;
    while (n != 0) {
        total += n % 10;
        n /= 10;
    }
    return total;
}

// Count digits
int countDigits(int n) {
    if (n == 0) return 1;
    n = abs(n);
    int count = 0;
    while (n != 0) {
        count++;
        n /= 10;
    }
    return count;
}
```

Java

```
// Sum of digits
int sumDigits(int n) {
    n = Math.abs(n);
    int total = 0;
    while (n != 0) {
        total += n % 10;
        n /= 10;
    }
    return total;
}

// Count digits
int countDigits(int n) {
    if (n == 0) return 1;
    n = Math.abs(n);
    int count = 0;
    while (n != 0) {
        count++;
        n /= 10;
    }
    return count;
}
```

Python

```
# Sum of digits
def sum_digits(n):
    n = abs(n)
    total = 0
    while n != 0:
        total += n % 10
        n //= 10
    return total

# Count digits
def count_digits(n):
    if n == 0: return 1
    n = abs(n)
    count = 0
    while n != 0:
        count += 1
        n //= 10
    return count
```

07 · Armstrong Number

EXAMPLE

153 $\rightarrow$ k = 3 digits

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153 \quad \checkmark$$

371 $\rightarrow$ k = 3 digits

$$3^3 + 7^3 + 1^3 = 27 + 343 + 1 = 371 \quad \checkmark$$

💡 KEY INSIGHT

A number = sum of each digit raised to power k

k = number of digits in that number

Step 1: count digits $\rightarrow$ k

Step 2: sum each digit^k using while loop

Step 3: compare total with original number

🕒 Time: $O(\log n)$ 📦 Space: $O(1)$

PSEUDOCODE

```
function isArmstrong(n):
```

```
    original = n
```

```
    k        = countDigits(n)
```

```
    total    = 0
```

```
    temp     = n
```

```
    while temp != 0:
```

```
        digit = temp % 10
```

```
        pw    = 1
```

```
        for i = 1 to k:
```

```
            pw = pw * digit     $\leftarrow \text{digit}^k$ 
```

```
            total = total + pw
```

```
            temp  = temp / 10
```

```
    return total == original
```

```
// Known: 1, 153, 370, 371, 407, 1634
```

Armstrong Number — C++ · Java · Python

C++

```
int countDigits(int n) {
    int c = 0;
    while (n != 0) { c++; n /= 10;
    }
    return c;
}

bool isArmstrong(int n) {
    int original = n;
    int k        = countDigits(n);
    int total     = 0;
    int temp      = n;

    while (temp != 0) {
        int digit = temp % 10;
        int pw     = 1;
        for (int i = 0; i < k; i++)
            pw *= digit;
        total += pw;
        temp  /= 10;
    }
    return total == original;
}
```

Java

```
int countDigits(int n) {
    int c = 0;
    while (n != 0) { c++; n /= 10;
    }
    return c;
}

boolean isArmstrong(int n) {
    int original = n;
    int k        = countDigits(n);
    int total     = 0;
    int temp      = n;

    while (temp != 0) {
        int digit = temp % 10;
        int pw     = 1;
        for (int i = 0; i < k; i++)
            pw *= digit;
        total += pw;
        temp  /= 10;
    }
    return total == original;
}
```

Python

```
def count_digits(n):
    c = 0
    while n != 0:
        c += 1
        n //= 10
    return c

def is_armstrong(n):
    original = n
    k        = count_digits(n)
    total     = 0
    temp      = n

    while temp != 0:
        digit = temp % 10
        pw     = 1
        for i in range(k):
            pw *= digit
        total += pw
        temp  //= 10

    return total == original
```


08 · Palindrome Number

EXAMPLE

```
isPalindrome(121)?  
reverse(121) = 121 → 121 == 121 ✓  
  
isPalindrome(1221)?  
reverse(1221) = 1221 → 1221==1221 ✓
```

💡 KEY INSIGHT

Reverse the number, compare with original
Negative numbers are never palindromes
If reversed == original → palindrome
Use long for reversed to avoid overflow

📦 Time: $O(\log n)$ 📦 Space: $O(1)$

PSEUDOCODE

```
function isPalindrome(n):  
    // Negatives are never palindromes  
    if n < 0: return False  
  
    original = n  
    rev      = 0  
  
    while n != 0:  
        digit = n % 10  
        rev   = rev * 10 + digit  
        n     = n / 10  
  
    return rev == original  
  
// Trace: n = 121  
// d=1 → rev=1,   n=12  
// d=2 → rev=12,  n=1  
// d=1 → rev=121, n=0  
// rev==original → True ✓
```

Palindrome Number — C++ · Java · Python

C++

```
bool isPalindrome(int n) {
    if (n < 0) return false;

    int original = n;
    long rev      = 0;

    while (n != 0) {
        int digit = n % 10;
        rev = rev * 10 + digit;
        n   = n / 10;
    }
    return rev == original;
}

// isPalindrome(121)  → true
// isPalindrome(1221) → true
// isPalindrome(123)  → false
// isPalindrome(-121) → false
```

Java

```
boolean isPalindrome(int n) {
    if (n < 0) return false;

    int original = n;
    long rev      = 0;

    while (n != 0) {
        int digit = n % 10;
        rev = rev * 10 + digit;
        n   = n / 10;
    }
    return rev == original;
}

// isPalindrome(121)  → true
// isPalindrome(1221) → true
// isPalindrome(123)  → false
// isPalindrome(-121) → false
```

Python

```
def is_palindrome(n):
    if n < 0:
        return False

    original = n
    rev      = 0

    while n != 0:
        digit = n % 10
        rev   = rev * 10 + digit
        n     = n // 10

    return rev == original

# is_palindrome(121)  → True
# is_palindrome(1221) → True
# is_palindrome(123)  → False
# is_palindrome(-121) → False
```

09 · Perfect Number

EXAMPLE

`isPerfect(28)?`

Divisors of 28: 1, 2, 4, 7, 14

Sum = 1+2+4+7+14 = 28 = n ✓

`isPerfect(6)?`

Divisors: 1, 2, 3

💡 KEY INSIGHT

Sum of all proper divisors equals the number

Proper divisors: all divisors excluding n itself

Check from 1 to $\sqrt{n}$ — add both i and n/i

Do not add n itself; always add 1 separately

🕒 Time: $O(\sqrt{n})$ 📦 Space: $O(1)$

PSEUDOCODE

```
function isPerfect(n):
```

```
    if n <= 1: return False
```

```
    sumDiv = 1 // 1 is always a divisor
```

```
    i = 2
```

```
    while i * i <= n:
```

```
        if n % i == 0:
```

```
            sumDiv = sumDiv + i
```

```
            if i != n / i:
```

```
                sumDiv = sumDiv + (n / i)
```

```
        i = i + 1
```

```
    return sumDiv == n
```

```
// Known: 6, 28, 496, 8128, 33550336
```

```
// Why i != n/i check?
```

```
// Avoids double-counting when i == n/i
```

```
// e.g. n=36, i=6: 36/6=6 — same!
```

Perfect Number — C++ · Java · Python

C++

```
bool isPerfect(int n) {
    if (n <= 1) return false;

    int sumDiv = 1;

    for (int i = 2; (long long)i *
i <= n; i++) {
        if (n % i == 0) {
            sumDiv += i;
            if (i != n / i)
                sumDiv += n / i;
        }
    }
    return sumDiv == n;
}

// isPerfect(6)    → true
// isPerfect(28)   → true
// isPerfect(12)   → false
```

Java

```
boolean isPerfect(int n) {
    if (n <= 1) return false;

    int sumDiv = 1;

    for (int i = 2; (long) i * i <=
n; i++) {
        if (n % i == 0) {
            sumDiv += i;
            if (i != n / i)
                sumDiv += n / i;
        }
    }
    return sumDiv == n;
}

// isPerfect(6)    → true
// isPerfect(28)   → true
// isPerfect(12)   → false
```

Python

```
def is_perfect(n):
    if n <= 1:
        return False

    sum_div = 1
    i = 2
    while i * i <= n:
        if n % i == 0:
            sum_div += i
            if i != n // i:
                sum_div += n // i
        i += 1

    return sum_div == n

# is_perfect(6)    → True
# is_perfect(28)   → True
# is_perfect(12)   → False
```

10 · Strong Number

EXAMPLE

```
isStrong(145)?  
1! + 4! + 5!  
= 1 + 24 + 120 = 145 ✓
```

```
isStrong(2)?  
2! = 2 → 2 == 2 ✓
```

💡 KEY INSIGHT

A number whose sum of factorial of its digits equals the number itself

Step 1: precompute factorials 0! to 9!

Step 2: extract each digit, add its factorial

Step 3: compare total with original

🕒 Time: $O(\log n)$ 📦 Space: $O(1)$

PSEUDOCODE

```
// Precompute factorials 0! to 9!
```

```
fact = [1,1,2,6,24,120,720,  
        5040,40320,362880]
```

```
function isStrong(n):
```

```
    original = n
```

```
    total    = 0
```

```
    while n != 0:
```

```
        digit = n % 10
```

```
        total = total + fact[digit]
```

```
        n     = n / 10
```

```
    return total == original
```

```
// Trace: isStrong(145)
```

```
// d=5 → total=120, n=14
```

```
// d=4 → total=144, n=1
```

```
// d=1 → total=145, n=0
```

```
// 145==145 → True ✓
```

Strong Number — C++ · Java · Python

C++

```
int fact[] = {1, 1, 2, 6, 24, 120,
              720, 5040, 40320,
              362880};
```

```
bool isStrong(int n) {
    int original = n;
    int total    = 0;

    while (n != 0) {
        int digit = n % 10;
        total    += fact[digit];
        n        /= 10;
    }
    return total == original;
}
```

```
// isStrong(1)   → true
// isStrong(2)   → true
// isStrong(145) → true
// isStrong(123) → false
```

Java

```
int[] fact = {1, 1, 2, 6, 24, 120,
              720, 5040, 40320,
              362880};
```

```
boolean isStrong(int n) {
    int original = n;
    int total    = 0;

    while (n != 0) {
        int digit = n % 10;
        total    += fact[digit];
        n        /= 10;
    }
    return total == original;
}
```

```
// isStrong(1)   → true
// isStrong(2)   → true
// isStrong(145) → true
// isStrong(123) → false
```

Python

```
fact = [1, 1, 2, 6, 24, 120,
        720, 5040, 40320, 362880]
```

```
def is_strong(n):
    original = n
    total    = 0

    while n != 0:
        digit = n % 10
        total += fact[digit]
        n     //= 10

    return total == original
```

```
# is_strong(1)   → True
# is_strong(2)   → True
# is_strong(145) → True
# is_strong(123) → False
```

11 · Automorphic Number

EXAMPLE

`isAutomorphic(25)?`

$25^2 = 625 \rightarrow$ ends in 25 ✓

`isAutomorphic(76)?`

$76^2 = 5776 \rightarrow$ ends in 76 ✓

💡 KEY INSIGHT

A number whose square ends with that number
Compare digits of n and n^2 from right to left
Extract last digit of both and check equality
Remove last digit of both; stop when $n=0$

□ **Time: $O(\log n)$** 📦 **Space: $O(1)$**

PSEUDOCODE

```
function isAutomorphic(n):
```

```
    sq  = n * n
```

```
    temp = n
```

```
    while temp != 0:
```

```
        // Last digit must match
```

```
        if sq % 10 != temp % 10:
```

```
            return False
```

```
            sq  = sq  / 10
```

```
            temp = temp / 10
```

```
    return True
```

```
// Trace: isAutomorphic(25)
```

```
// sq=625, temp=25
```

```
// 625%10=5, 25%10=5 ✓ → sq=62, t=2
```

```
// 62%10=2, 2%10=2 ✓ → sq=6, t=0
```

```
// temp=0 → return True ✓
```

Automorphic Number — C++ · Java · Python

C++

```
bool isAutomorphic(int n) {
    long sq  = (long) n * n;
    int temp = n;

    while (temp != 0) {
        if (sq % 10 != temp % 10)
            return false;
        sq /= 10;
        temp /= 10;
    }
    return true;
}

// isAutomorphic(5) → true  (25
ends 5)
// isAutomorphic(25) → true  (625
ends 25)
// isAutomorphic(76) → true  (5776
ends 76)
// isAutomorphic(7) → false (49
ends 9)
```

Java

```
boolean isAutomorphic(int n) {
    long sq  = (long) n * n;
    int temp = n;

    while (temp != 0) {
        if (sq % 10 != temp % 10)
            return false;
        sq /= 10;
        temp /= 10;
    }
    return true;
}

// isAutomorphic(5) → true
// isAutomorphic(25) → true
// isAutomorphic(76) → true
// isAutomorphic(7) → false
```

Python

```
def is_automorphic(n):
    sq  = n * n
    temp = n

    while temp != 0:
        if sq % 10 != temp % 10:
            return False
        sq //= 10
        temp //= 10

    return True

# is_automorphic(5) → True
# is_automorphic(25) → True
# is_automorphic(76) → True
# is_automorphic(7) → False
```


Practice Problems — Fundamentals

⚡ EASY — Start Here

Reverse Integer

LC #7

Reverse digits + check INT_MAX overflow

Palindrome Number

LC #9

Reverse the number, compare with original

Count Primes

LC #204

Sieve of Eratosthenes — count True entries

Happy Number

LC #202

Sum of digit squares — detect cycle (slow/fast)

Ugly Number

LC #263

Divide by 2, 3, 5 until 1; check if $n=1$

Number of Steps to Zero

LC #1342

Even: $n/2$. Odd: $n-1$. Count total steps

MEDIUM — Next Level

GCD of Strings

LC #1071

$\text{len}(s1+s2) == \text{len}(s2+s1) \rightarrow \text{gcd-length prefix}$

Find GCD of Array

LC #1979

Apply gcd on all array elements iteratively

Self Dividing Numbers

LC #728

Each digit $\neq 0$ and divides the number

Nth Digit

LC #400

Find range, then exact number and digit index

Sum of Digits = Product

LC #1295

Count numbers where digit sum equals product

Check if Num Sum of Powers

LC #2999

Armstrong-style: find x where x^n sums to num



DSA DOJO BATCH 4

@sansaura_

instagram · youtube
sansaura.in

FUNDAMENTALS DONE.

These are your weapons. Now sharpen them.

- ✓ **Code GCD, LCM, isPrime from scratch — no looking**
- ✓ Trace every algorithm on paper with a 3-digit number
- ✓ Solve LC #7, #9, #202, #204 this week
- ✓ Post your solution in the Dojo group

⚡ QUICK REFERENCE

```
GCD: while b!=0 { t=b; b=a%b; a=t; } return a
LCM: (a / gcd(a,b)) * b
Prime: check i*i <= n in loop
Reverse / Digits: digit=n%10; n/=10
Armstrong: sum(digit^k)==n | Perfect: sumDiv==n
Strong: sum(fact[digit])==n | Automorphic: sq%10==n%10
```